# UNIVERSITI TEKNOLOGI MALAYSIA

DEPARTMENT OF COMPUTER SCIENCE
FACULTY OF COMPUTING

SECP3843 - SPECIAL TOPIC IN DATA ENGINEERING

# Tutorial 4 : Generative AI



| NAME | MATRIC NO |
|---|---|
| MUHAMMAD NAIM BIN ABDULLAH | A23CS0134 |
| MUHAMMAD ADAM BIN RAZALI | A23CS0116 |
| AFIF SHAQIR IRFAN BIN ARQAM | A23CS0204 |



*Submitted to: Department of Computer Science, Faculty of Computing, UTM*

# 1. Introduction

This tutorial demonstrates the integration of **Generative AI (Claude)** into a real-world data engineering pipeline. The project uses the *Dirty Cafe Sales* dataset — a synthetic dataset from Kaggle containing 10,000 transactions — intentionally seeded with data quality issues such as missing values, invalid `ERROR`/`UNKNOWN` placeholders, inconsistent date formats, and duplicate rows.

The goal is to build a complete **ETL (Extract, Transform, Load)** pipeline with AI assistance, where Claude generates each stage of the pipeline code from natural-language prompts. This explores how AI can accelerate data engineering tasks while still requiring human oversight for validation and correctness.

## 1.1 Objectives

- Design and implement a 4-stage ETL data pipeline for cafe sales data.
- Leverage Claude (Generative AI) to generate Python code for each pipeline stage.
- Identify and resolve data quality issues (missing values, duplicates, type errors).
- Load cleaned data into a SQLite database and generate business insights.
- Demonstrate prompt engineering as a practical data engineering skill.

## 1.2 Dataset Overview

The *Dirty Cafe Sales* dataset contains 2,020 raw records (after deduplication: **2,000 rows**) across 8 columns. Key data quality issues present in the raw file:

| Column | Issue Type | Handling Strategy |
|---|---|---|
| Item | NULL / UNKNOWN placeholders | Fill with 'Unknown' |
| Quantity | ERROR strings, NaN | Convert to numeric; fill with median |
| Price Per Unit | ERROR strings, NaN | Convert to numeric; fill with median |
| Total Spent | Missing / inconsistent | Recalculate from Qty x Price; fill median |
| Transaction Date | Unparseable / malformed | Parse with coerce; drop invalid rows |
| Payment Method | UNKNOWN placeholders | Fill with 'Unknown' |
| Location | UNKNOWN placeholders | Fill with 'Unknown' |

## 2. Pipeline Structure

The pipeline follows a classic **ETL architecture** broken into four sequential Python scripts, each with a single responsibility. An orchestrator script (`run_pipeline.py`) chains them together and halts execution if any stage fails.

**Stage 1 — Ingest**

`01_ingest.py`

Load dirty_cafe_sales.csv into a pandas DataFrame.

Profile data: shape, dtypes, missing values, duplicates, unique values.

↓

**Stage 2 — Transform**

`02_transform.py`

Remove duplicates. Replace ERROR/UNKNOWN with NaN.

Fix numeric types. Recalculate Total Spent.

Fill missing values (median / 'Unknown').

Parse dates, drop unparseable. Add Day of Week & Transaction Month columns.

↓

**Stage 3 — Load**

`03_load.py`

Write cleaned DataFrame into SQLite database (cafe_sales.db)

as the cafe_sales table, replacing any existing data.

↓

**Stage 4 — Analyze**

`04_analyze.py`

Run SQL queries: total revenue, by item, by day,

by payment method, by location.

Generate bar chart (revenue_by_item.png).

### 2.1 Orchestrator: run_pipeline.py

The orchestrator script runs all four stages sequentially using `subprocess.run()`. If any stage returns a non-zero exit code, the pipeline stops immediately and reports the failure, preventing downstream

stages from running on bad data.

```
STAGES = ['01_ingest.py', '02_transform.py', '03_load.py', '04_analyze.py']
for stage in STAGES:
    result = subprocess.run([sys.executable, stage])
    if result.returncode != 0:
        print(f'Stage {stage} failed. Stopping.')
        sys.exit(1)
```

## 2.2 Outputs

| File | Description |
|---|---|
| cleaned_cafe_sales.csv | Fully cleaned dataset with engineered columns |
| cafe_sales.db | SQLite database with cafe_sales table (1,824 rows) |
| revenue_by_item.png | Bar chart: total revenue per menu item |

# 3. How Claude (AI) Was Used in This Pipeline

Claude was used as a **code generation assistant** throughout this project. Each pipeline stage was built by issuing a structured natural-language prompt to Claude, reviewing the generated code, running it, and iteratively refining based on output. This section documents each prompt and the resulting code logic.

## Prompt 1: Dataset Selection

> ***Prompt given to Claude:*** *"Find a suitable Kaggle dataset for practicing an ETL pipeline with realistic data cleaning challenges — missing values, invalid entries, inconsistent formats."*

**Result:** Claude recommended the Dirty Cafe Sales dataset (10,000 synthetic café transactions with intentional data quality issues: missing values, ERROR/UNKNOWN placeholders, inconsistent dates, and duplicate rows).

---

## Prompt 2: Ingestion Script (01_ingest.py)

> ***Prompt given to Claude:*** *"Write a Python script that loads dirty_cafe_sales.csv into a pandas DataFrame and prints a data quality profile: shape, column dtypes, missing value counts, sample rows, and unique values for categorical columns."*

**Result:** Claude generated load_raw_data() and profile_data() functions. The profile revealed: 2,020 rows with 20 duplicates, and hundreds of ERROR/UNKNOWN strings across numeric and categorical columns.

**Generated code excerpt:**

```
def load_raw_data(path: str) -> pd.DataFrame:
    df = pd.read_csv(path)
    return df
```

```
def profile_data(df: pd.DataFrame) -> None:
    print(f'Rows: {df.shape[0]}, Columns: {df.shape[1]}')
    print(df.dtypes)
    print(df.isna().sum())
    print('Unique Item values:', df['Item'].unique())
```

---

## Prompt 3: Transformation Script (02_transform.py)

> ***Prompt given to Claude:*** *"Write a data cleaning pipeline that: removes duplicate rows, replaces ERROR and UNKNOWN strings with NaN, converts Quantity/Price/Total Spent to numeric, recalculates missing Total Spent from Quantity x Price Per Unit, fills remaining numeric NaNs with column median and categorical NaNs with 'Unknown', parses Transaction Date (dropping unparseable rows), and adds Day of Week and Transaction Month derived columns."*

**Result:** Claude produced seven focused helper functions composed inside a single transform() function. The pipeline reduced 2,020 raw rows to 1,824 fully clean rows with zero missing values.

**Generated code excerpt:**

```
def transform(df):
    df = remove_duplicates(df)          # 2020 -> 2000 rows
    df = standardize_placeholders(df)   # ERROR/UNKNOWN -> NaN
    df = fix_numeric_columns(df)        # coerce to float
```

```
    df = recalculate_total_spent(df)    # Qty * Price where missing
    df = fill_missing_values(df)        # median / 'Unknown'
    df = clean_dates(df)                # 2000 -> 1824 rows
    df = add_derived_columns(df)        # Day of Week, Month
    return df.reset_index(drop=True)
```

## Prompt 4: Loading Script (03_load.py)

> ***Prompt given to Claude:*** *"Write a script that accepts a cleaned pandas DataFrame and loads it into a SQLite database file (cafe_sales.db) as a table named 'cafe_sales', replacing any existing table."*

**Result:** Claude generated a concise load_to_sqlite() function using pandas to_sql() with if_exists='replace'. The function accepts configurable db_file and table_name parameters for reusability.

**Generated code excerpt:**

```
def load_to_sqlite(df, db_file=DB_FILE, table_name=TABLE_NAME):
    conn = sqlite3.connect(db_file)
    df.to_sql(table_name, conn, if_exists='replace', index=False)
    conn.close()
    print(f'Loaded {len(df)} rows into {table_name}')
```

## Prompt 5: Analysis Script (04_analyze.py)

> ***Prompt given to Claude:*** *"Write SQL queries against the SQLite database to calculate: total revenue, revenue and units sold by item, revenue by day of week, revenue and transaction count by payment method, and by location. Also generate a bar chart of revenue by item saved as revenue_by_item.png."*

**Result:** Claude generated five SQL queries using GROUP BY and SUM aggregations, wrapped in a run_query() helper. The chart uses matplotlib with a coffee-brown colour palette (#6f4e37) appropriate for a cafe context.

**Generated code excerpt:**

```
SELECT Item,
       ROUND(SUM([Total Spent]), 2) AS revenue,
       SUM(Quantity) AS units_sold
FROM cafe_sales
GROUP BY Item
ORDER BY revenue DESC
```

## Prompt 6: Orchestrator Script (run_pipeline.py)

> ***Prompt given to Claude:*** *"Combine all four stage scripts into a single run_pipeline.py that executes them in order using subprocess, stops immediately if any stage fails, and prints a success message with the list of output files on completion."*

**Result:** Claude generated a minimal orchestrator that iterates the STAGES list and calls subprocess.run() for each, checking returncode != 0 to halt on failure.

# 4. Results & Analysis

## 4.1 Data Cleaning Summary

| Metric | Before Cleaning | After Cleaning |
|---|---|---|
| Total rows | 2,020 | 1,824 |
| Duplicate rows | 20 | 0 |
| Missing / invalid values | 200–400 per column | 0 |
| Columns | 8 | 10 (+ Day of Week, Month) |

## 4.2 Revenue Analysis

After loading the 1,824 clean rows into SQLite, the analysis queries produced the following business insights. **Total revenue across all transactions: RM 15,777.50.**

### Revenue by Item

| Item | Revenue (RM) | Units Sold |
|---|---|---|
| Unknown | 4,364.00 | 1,492 |
| Smoothie | 2,214.50 | 563 |
| Salad | 1,968.00 | 465 |
| Sandwich | 1,848.00 | 504 |
| Cake | 1,397.00 | 483 |
| Juice | 1,376.00 | 478 |
| Coffee | 1,002.00 | 464 |
| Tea | 893.00 | 481 |
| Cookie | 715.00 | 505 |

**Key insight:** The 'Unknown' item category contributes the highest revenue (RM 4,364.00) due to a large number of transactions where the item could not be recovered — this is a data quality artifact from missing item names being imputed. Among identifiable items, **Smoothie** (RM 2,214.50) is the top performer, followed by Salad and Sandwich. Cookie generates the lowest revenue per transaction despite high unit volume (505 units), reflecting its lower price point.

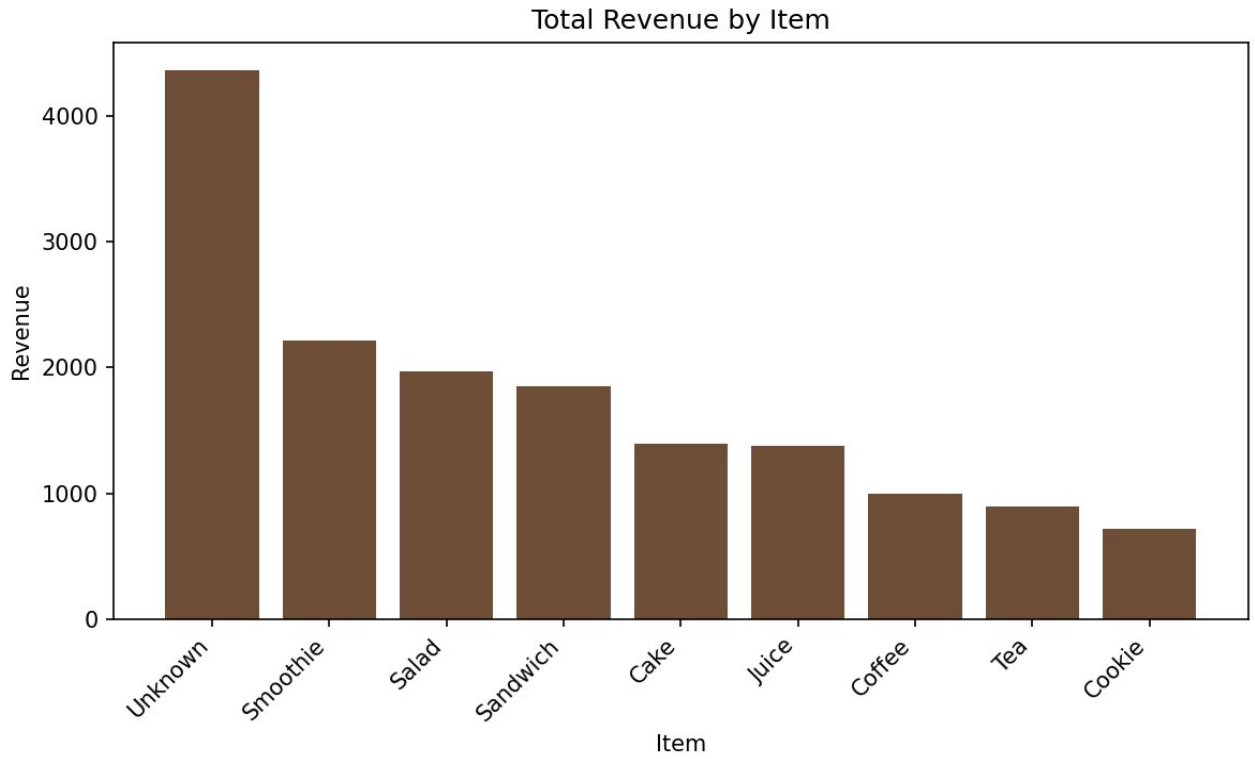


*Figure 1: Total Revenue by Item (generated by 04_analyze.py)*

### Revenue by Day of Week

| Day | Revenue (RM) |
|---|---|
| Monday | 2,387.50 |
| Saturday | 2,336.00 |
| Thursday | 2,326.00 |
| Friday | 2,293.50 |
| Tuesday | 2,272.50 |
| Sunday | 2,091.00 |
| Wednesday | 2,071.00 |

**Key insight:** Revenue is relatively balanced across all days (RM 2,071 – RM 2,387). Monday has the highest revenue, which is slightly unexpected — this may reflect morning rush commuters. Wednesday and Sunday record the lowest revenue, suggesting quieter trading periods.

### Revenue by Payment Method & Location

| Payment Method | Revenue (RM) | Transactions |
|---|---|---|
| Unknown | 8,048.50 | 931 |
| Digital Wallet | 2,904.50 | 313 |
| Credit Card | 2,562.00 | 307 |
| Cash | 2,262.50 | 273 |

| Location | Revenue (RM) | Transactions |
|---|---|---|
| Unknown | 9,847.00 | 1,133 |
| Takeaway | 3,137.50 | 355 |
| In-store | 2,793.00 | 336 |

**Key insight:** A large proportion of payment method and location data is classified as 'Unknown' due to unrecoverable missing values — a direct result of the dataset's intentional data quality issues. Among known payment methods, Digital Wallet leads (RM 2,904.50), while Takeaway slightly outperforms In-store among known locations.

# 5. Reflection

### MUHAMMAD NAIM BIN ABDULLAH (A23CS0134)

Working on this pipeline showed how Claude can make data engineering much easier and faster. Instead of writing all the code by hand, Claude helped build each step of the pipeline — from reading the raw data, to cleaning it, to saving it in a database, and finally producing useful results. This is similar to what the tutorial showed, where AI can understand pipelines and help fix or build them just by chatting. Even though Claude did most of the coding, the results still needed to be checked to make sure the cleaning steps made sense, such as filling in missing values correctly. Overall, this shows that AI can save a lot of time, but human guidance and checking are still important.

### MUHAMMAD ADAM BIN RAZALI (A23CS0116)

This tutorial gave me a clearer understanding of how Generative AI fits into a practical data engineering workflow. Before this, I assumed writing ETL pipelines required significant manual effort for every function. Working with Claude changed that perspective — by describing what each stage needed to do in plain language, the code was generated quickly and correctly for most cases. What stood out to me was how important it is to give precise prompts. Vague requests produced generic code, while specific prompts (mentioning exact column names, strategies for missing values, expected output format) resulted in production-ready scripts with minimal editing. This experience reinforced that prompt engineering is a real and valuable skill in modern data work, not just a shortcut.

### AFIF SHAQIR IRFAN BIN ARQAM (A23CS0204)

Building this pipeline with Claude gave me a hands-on appreciation of how AI can genuinely change the speed of development in data engineering. My responsibility was understanding how each generated script connected to the next and ensuring the overall pipeline ran end-to-end without errors in VS Code. One key takeaway is that AI does not eliminate the need for data engineering knowledge — it amplifies it. When Claude generated the transformation script, I still needed to understand why median imputation was chosen over mean, why rows with unparseable dates should be dropped rather than filled, and what the 'Unknown' category in the results actually means for the business. Without that understanding, errors would have gone unnoticed and the analysis would have been misleading. This tutorial demonstrated that the future of data engineering will involve AI as a collaborator — but the engineer's judgement, domain understanding, and ability to validate outputs remain indispensable.